

SQL Complete Learning Roadmap

This document is a complete, practical, and interview-ready guide to SQL. It explains the theory behind each concept, shows the correct syntax, and gives simple examples you can run in PostgreSQL or any standard SQL database.

1. SQL Fundamentals

What is SQL?

SQL (Structured Query Language) is the standard language used to store, retrieve, update, and manage data in relational databases.

Why SQL is important

- It is used in almost every data system.
- It helps query large datasets efficiently.
- It is the foundation of analytics, reporting, backend development, and data engineering.

Main categories of SQL commands

1. DDL – defines database objects
2. DML – manipulates data
3. DQL – reads data
4. DCL – controls access
5. TCL – manages transactions

DDL

Used to create and modify structure of data.

```
CREATE TABLE employees (  
    id INT PRIMARY KEY,  
    name VARCHAR(100),  
    salary DECIMAL(10,2)  
);  
  
ALTER TABLE employees ADD COLUMN department VARCHAR(50);  
DROP TABLE employees;
```

DML

Used to insert, update, or delete rows.

```
INSERT INTO employees (id, name, salary, department)
VALUES (1, 'Aman', 50000, 'HR');
```

```
UPDATE employees
SET salary = 55000
WHERE id = 1;
```

```
DELETE FROM employees
WHERE id = 1;
```

DQL

Used to fetch data.

```
SELECT * FROM employees;
SELECT name, salary FROM employees WHERE salary > 40000;
```

DCL

Used to grant and revoke permissions.

```
GRANT SELECT ON employees TO analyst;
REVOKE SELECT ON employees FROM analyst;
```

TCL

Used to manage transactions safely.

```
BEGIN;
UPDATE employees SET salary = 60000 WHERE id = 1;
COMMIT;

-- or
BEGIN;
UPDATE employees SET salary = 70000 WHERE id = 1;
ROLLBACK;
```

2. SQL Data Types

Data types define what kind of values a column can store.

Numeric types

- **INT** – whole numbers
- **BIGINT** – large integers
- **SMALLINT** – smaller integers
- **DECIMAL(p,s)** / **NUMERIC(p,s)** – exact decimal values
- **FLOAT** / **REAL** – approximate decimal values

Example:

```
CREATE TABLE products (  
    product_id INT,  
    price DECIMAL(10,2),  
    rating FLOAT  
);
```

Character types

- **CHAR(n)** – fixed length
- **VARCHAR(n)** – variable length
- **TEXT** – long text

Example:

```
CREATE TABLE customers (  
    customer_id INT,  
    first_name VARCHAR(50),  
    email TEXT  
);
```

Date and time types

- **DATE** – year-month-day
- **TIME** – time of day
- **TIMESTAMP** – date and time
- **INTERVAL** – duration

Example:

```
CREATE TABLE events (  
    event_id INT,  
    event_date DATE,  
    event_time TIME,  
    created_at TIMESTAMP  
);
```

Boolean and JSON

```
CREATE TABLE orders (  
    is_active BOOLEAN,  
    metadata JSONB  
);
```

3. SQL Operators

Operators are used to perform actions on values and expressions.

Arithmetic operators

```
SELECT 10 + 5, 10 - 5, 10 * 5, 10 / 5, 10 % 3;
```

Comparison operators

```
SELECT * FROM employees WHERE salary = 50000;  
SELECT * FROM employees WHERE salary != 50000;  
SELECT * FROM employees WHERE salary > 40000;  
SELECT * FROM employees WHERE salary >= 40000;
```

Logical operators

```
SELECT * FROM employees  
WHERE department = 'HR' AND salary > 40000;  
  
SELECT * FROM employees  
WHERE department = 'HR' OR department = 'IT';
```

```
SELECT * FROM employees  
WHERE NOT department = 'HR';
```

Special operators

- **IN** – checks membership
- **BETWEEN** – checks range
- **LIKE** – pattern matching
- **ILIKE** – case-insensitive LIKE
- **EXISTS** – checks subquery result

Example:

```
SELECT * FROM employees  
WHERE department IN ('HR', 'IT');  
  
SELECT * FROM employees  
WHERE salary BETWEEN 40000 AND 70000;  
  
SELECT * FROM employees  
WHERE name LIKE 'A%';
```

Set operators

- **UNION** – combines distinct rows
- **UNION ALL** – combines all rows
- **INTERSECT** – common rows
- **EXCEPT** – rows in first but not second

```
SELECT name FROM employees  
UNION  
SELECT name FROM managers;
```

4. Constraints

Constraints enforce rules on table data.

Common constraints

- **PRIMARY KEY** – unique and not null identifier
- **FOREIGN KEY** – links one table to another

- **UNIQUE** – no duplicates
- **NOT NULL** – value cannot be null
- **CHECK** – custom validation rule
- **DEFAULT** – default value if not provided

Example:

```
CREATE TABLE departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50) UNIQUE NOT NULL  
);  
  
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    dept_id INT,  
    salary DECIMAL(10,2) DEFAULT 0,  
    CONSTRAINT fk_emp_dept  
        FOREIGN KEY (dept_id) REFERENCES departments(dept_id),  
    CONSTRAINT chk_salary_positive CHECK (salary >= 0)  
);
```

5. SQL Clauses

Clauses are building blocks of SQL statements.

Important clauses

- **SELECT** – choose columns
- **FROM** – specify table
- **WHERE** – filter rows
- **GROUP BY** – group rows
- **HAVING** – filter grouped rows
- **ORDER BY** – sort output
- **LIMIT** – restrict number of rows
- **OFFSET** – skip rows
- **DISTINCT** – remove duplicates

Example:

```
SELECT department, AVG(salary) AS avg_salary  
FROM employees  
WHERE salary > 30000
```

```
GROUP BY department
HAVING AVG(salary) > 40000
ORDER BY avg_salary DESC
LIMIT 5;
```

6. Joins

Joins are used to combine rows from two or more tables.

Types of joins

- INNER JOIN – matching rows only
- LEFT JOIN – all rows from left, matching from right
- RIGHT JOIN – all rows from right
- FULL OUTER JOIN – all rows from both sides
- CROSS JOIN – every row with every row
- SELF JOIN – table joins with itself

Example:

```
SELECT e.name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
```

Left join example

```
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;
```

7. Subqueries

A subquery is a query written inside another query.

Types of subqueries

- Single-row subquery

- Multiple-row subquery
- Multiple-column subquery
- Correlated subquery
- Nested subquery

Example:

```
SELECT name
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

EXISTS vs IN

```
SELECT name
FROM employees e
WHERE EXISTS (
    SELECT 1 FROM departments d WHERE d.dept_id = e.dept_id
);
```

```
SELECT name
FROM employees
WHERE dept_id IN (1, 2, 3);
```

8. Common Table Expressions (CTE)

CTEs make complex queries easier to read.

Syntax

```
WITH cte_name AS (
    SELECT ...
)
SELECT * FROM cte_name;
```

Example


```
WITH high_salary AS (  
    SELECT * FROM employees WHERE salary > 50000  
)  
SELECT * FROM high_salary;
```

Recursive CTE

```
WITH RECURSIVE numbers(n) AS (  
    SELECT 1  
    UNION ALL  
    SELECT n + 1 FROM numbers WHERE n < 5  
)  
SELECT * FROM numbers;
```

9. Views

A view is a stored query that behaves like a table.

Benefits

- Simplifies complex queries
- Improves security
- Provides reusable logic

Example:

```
CREATE VIEW hr_employees AS  
SELECT emp_id, name, salary  
FROM employees  
WHERE department = 'HR';
```

Use it like a table:

```
SELECT * FROM hr_employees;
```

10. SQL Functions

Functions help transform, aggregate, and analyze data.

Aggregate functions

```
SELECT COUNT(*) AS total_employees,  
       AVG(salary) AS avg_salary,  
       SUM(salary) AS total_salary,  
       MIN(salary) AS min_salary,  
       MAX(salary) AS max_salary  
FROM employees;
```

String functions

```
SELECT UPPER(name), LOWER(name), LENGTH(name), TRIM(name)  
FROM employees;
```

Date functions

```
SELECT CURRENT_DATE, CURRENT_TIMESTAMP, EXTRACT(YEAR FROM created_at)  
FROM employees;
```

Conditional functions

```
SELECT name,  
       CASE  
         WHEN salary >= 70000 THEN 'High'  
         WHEN salary >= 40000 THEN 'Medium'  
         ELSE 'Low'  
       END AS salary_band  
FROM employees;
```

11. User-Defined Functions (UDFs)

UDFs allow you to create custom reusable logic.

Types

- **Scalar function** – returns one value
- **Table-valued function** – returns a table

Example:

```
CREATE FUNCTION get_bonus(salary DECIMAL)
RETURNS DECIMAL
AS $$
    SELECT salary * 0.10;
$$ LANGUAGE SQL;
```

12. Stored Procedures

A stored procedure is a named block of SQL logic stored in the database.

Example:

```
CREATE PROCEDURE increase_salary(emp_id INT, amount DECIMAL)
LANGUAGE SQL
AS $$
    UPDATE employees
    SET salary = salary + amount
    WHERE emp_id = emp_id;
$$;
```

Difference from function:

- Procedure may perform DML and return multiple outputs.
- Function usually returns a single value or table.

13. Triggers

A trigger automatically runs when a specified event happens.

Trigger types

- **BEFORE trigger**
- **AFTER trigger**

- **INSTEAD OF trigger**

Example:

```
CREATE TRIGGER log_salary_change
AFTER UPDATE ON employees
FOR EACH ROW
EXECUTE FUNCTION log_employee_update();
```

14. Window Functions

Window functions compute values over a set of rows related to the current row.

Common window functions

- ROW_NUMBER()
- RANK()
- DENSE_RANK()
- SUM() OVER()
- AVG() OVER()
- LEAD()
- LAG()

Example:

```
SELECT name,
       salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn
FROM employees;
```

Running total example

```
SELECT name, salary,
       SUM(salary) OVER (ORDER BY salary) AS running_total
FROM employees;
```

15. Transactions and ACID

A transaction is a unit of work that must be completed fully or not at all.

ACID properties

- Atomicity – all or nothing
- Consistency – data remains valid
- Isolation – concurrent transactions do not interfere badly
- Durability – committed data is permanent

Example:

```
BEGIN;  
UPDATE employees SET salary = salary + 5000 WHERE emp_id = 1;  
COMMIT;
```

If something fails:

```
BEGIN;  
UPDATE employees SET salary = salary + 5000 WHERE emp_id = 1;  
ROLLBACK;
```

16. Indexes

Indexes improve the speed of data retrieval.

Types of indexes

- B-tree index – default and common
- Unique index
- Composite index
- Partial index
- Hash index
- GIN / GiST / BRIN – advanced PostgreSQL indexes

Example:

```
CREATE INDEX idx_emp_salary ON employees(salary);  
CREATE INDEX idx_emp_dept_salary ON employees(dept_id, salary);
```

When to use indexes

- On columns used heavily in WHERE
- On join columns
- On columns used in sorting

17. Partitioning and Sharding

Partitioning

Splits one large table into smaller logical parts.

Example:

```
CREATE TABLE orders_2026 (  
    order_id INT,  
    order_date DATE  
) PARTITION OF orders  
FOR VALUES FROM ('2026-01-01') TO ('2027-01-01');
```

Sharding

Distributes data across multiple servers or databases for scale.

Types:

- Horizontal sharding
- Vertical sharding
- Key-based or range-based sharding

18. Query Optimization

Query optimization improves performance.

Useful techniques

- Use indexes appropriately
- Avoid SELECT * when not needed
- Use EXPLAIN to inspect execution plans
- Normalize data where necessary

Example:

EXPLAIN ANALYZE

```
SELECT name FROM employees WHERE salary > 50000;
```

19. PostgreSQL Administration

VACUUM

Reclaims storage and updates visibility maps.

```
VACUUM;  
VACUUM FULL;
```

ANALYZE

Updates table statistics for the optimizer.

```
ANALYZE employees;
```

REINDEX

Rebuilds an index.

```
REINDEX INDEX idx_emp_salary;
```

20. Temporary Objects

Temporary objects exist only for the current session.

```
CREATE TEMP TABLE temp_employees AS  
SELECT * FROM employees WHERE salary > 60000;
```

21. Advanced PostgreSQL Topics

JSON and JSONB

```
SELECT metadata->>'department' FROM orders;
```

Arrays

```
CREATE TABLE tags (id INT, names TEXT[]);
```

22. Important SQL Differences

DROP vs DELETE vs TRUNCATE

- DROP removes the table structure completely.
- DELETE removes rows one by one.
- TRUNCATE removes all rows quickly but keeps the table.

UNION vs UNION ALL

- UNION removes duplicates.
- UNION ALL keeps duplicates.

WHERE vs HAVING

- WHERE filters rows before grouping.
- HAVING filters after grouping.

RANK vs DENSE_RANK

- RANK skips ranking numbers after ties.
- DENSE_RANK keeps consecutive ranks.

View vs Materialized View

- View is virtual and updates dynamically.
 - Materialized View stores the result physically.
-

23. Database Design Basics

Normalization

Normalization reduces redundancy and improves data integrity.

- 1NF – atomic values
- 2NF – no partial dependency
- 3NF – no transitive dependency
- BCNF – stronger version of 3NF

Denormalization

Used for performance in analytics systems.

Star schema

A fact table connected to dimension tables.

Snowflake schema

A normalized form of star schema.

24. Interview-Oriented SQL Practice

Top-N query

```
SELECT *  
FROM employees  
ORDER BY salary DESC  
LIMIT 5;
```

Running total

```
SELECT name, salary,  
       SUM(salary) OVER (ORDER BY salary)  
FROM employees;
```

Duplicate detection

```
SELECT name, COUNT(*)  
FROM employees  
GROUP BY name  
HAVING COUNT(*) > 1;
```

Gap and island problem

Used to detect consecutive ranges or missing values in a dataset.

25. Best Practices for SQL

1. Use meaningful table and column names.
2. Prefer explicit column lists instead of SELECT *.
3. Use indexes carefully.
4. Write readable queries with aliases and CTEs.
5. Test with EXPLAIN ANALYZE for performance.
6. Use constraints to protect data quality.
7. Always use transactions for critical updates.

Quick Tips

- Always write queries in a readable format with line breaks.
- Use aliases like e for employees and d for departments to keep SQL short.
- When in doubt, start with SELECT * FROM table LIMIT 10 to verify your data.
- Use comments in long queries for easier understanding.
- Practice writing the same logic using JOIN, CTE, and subquery forms.

Common Interview Questions

1. What is the difference between WHERE and HAVING?
2. What is the difference between UNION and UNION ALL?
3. What is the difference between DELETE, DROP, and TRUNCATE?
4. What are primary keys and foreign keys?
5. What is a join? Explain INNER JOIN vs LEFT JOIN.
6. What is the difference between RANK and DENSE_RANK?
7. What is a CTE and when should you use it?
8. How do indexes improve performance?
9. What is normalization and why is it important?
10. What is a window function? Give one example.

Study Suggestions

- Practice 5 basic SELECT queries daily.
- Solve 1 join problem, 1 subquery problem, and 1 window function problem every day.
- Learn SQL by writing queries on real datasets instead of only reading theory.
- Revise the difference between DDL, DML, DCL, and TCL regularly.
- Make flashcards for important concepts like constraints, joins, indexes, and transactions.
- Use PostgreSQL or any SQL editor to execute your queries and verify output.

Quick Revision Summary

- SQL is used to manage relational databases.
- DDL creates structure, DML modifies data, DQL reads data.
- Constraints protect data quality.
- Joins connect related data.
- Subqueries and CTEs help solve complex logic.
- Window functions are powerful for ranking and analytics.
- Indexes and optimization improve performance.
- Transactions ensure safe database updates.

This roadmap gives you the theory, syntax, and examples needed to learn SQL step by step and prepare for interviews and real projects.